

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA14

COURSE NAME: COMPILER DESIGN

COURSE OUTCOME COVERED

CO5: *Develop code optimization and Code Generation using DAG representation with data flow analysis to produce optimized target code. (BL3)*

Assessment Tool Weight-age: 40%

QUESTIONS: 5

Total Marks:50

Annexure A

SIMULATION TASK

Code of Conduct:

*I M.VINAY(192311146) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

M.VINAY

Signature of the student:

Criteria	Weight age (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Conceptual Understanding	20%	Demonstrates strong understanding of underlying concepts in the simulation	Shows good understanding with minor gaps	Partial understanding; some misconceptions	Lacks understanding of key concepts

Model Design	25%	Develops accurate, well-structured simulation/model independently	Develops correct model with minor errors	Model is incomplete or requires guidance	Unable to develop a proper model
Tool Usage	20%	Uses simulation tools effectively with correct setup and execution	Uses tools with minor errors or guidance	Limited ability to use tools properly	Unable to use simulation tools
Analysis of Results	20%	Provides clear, logical analysis and meaningful interpretation of results	Analysis is reasonable with minor gaps	Superficial analysis; limited interpretation	Unable to analyze or interpret results
Documentation	15%	Presents results clearly with proper documentation and structured reporting	Documentation is adequate with minor issues	Poorly organized or incomplete documentation	No proper documentation or presentation

Annexure A

SIMULATION TASK

QUESTIONS

1. Given the three-address code sequence:

$t1 = x * 1$ $t2 =$

$t1 + y$ $t3 = t2 -$

0

$t4 = t3 * 0$

if $t4 \neq 0$ goto L1 $t5 =$

$y + y$

Simulate the **peephole optimization** process on this sequence. Produce the optimized instruction sequence and justify each optimization step applied, including algebraic simplification, constant folding, and dead code elimination.

1A) . Peephole Optimization

Given Code

$t1 = x * 1$

$t2 = t1 + y$

$t3 = t2 - 0$

$t4 = t3 * 0$

if $t4 \neq 0$ goto L1

$t5 = y + y$

Optimization Steps

Step 1: Algebraic Simplification

$t1 = x * 1 \rightarrow t1 = x$

Step 2: Substitute t1

$t2 = x + y$

Step 3: Remove Addition/Subtraction by Zero

$t3 = t2 - 0 \rightarrow t3 = t2$

Step 4: Multiplication by Zero

$t4 = t3 * 0 \rightarrow t4 = 0$

Step 5: Constant Folding

if $t4 \neq 0$ goto L1

if $0 \neq 0$ goto L1

Condition is always false.

Step 6: Dead Code Elimination

Remove conditional jump.

Optimized Code

$t2 = x + y$

$t5 = y + y$

2. Partition the following intermediate code into **basic blocks** and construct the corresponding **control flow graph**. Identify all leader statements and mention the rules used to identify them:

(1) $x = a + b$

(2) $y = x * c$

(3) if $y > 50$ goto 6

(4) $z = y - d$

(5) goto 7

(6) $z = y + d$

(7) $w = z * 2$

2A) Basic Blocks and Control Flow Graph

Intermediate Code

- (1) $x = a + b$
- (2) $y = x * c$
- (3) if $y > 50$ goto 6
- (4) $z = y - d$
- (5) goto 7
- (6) $z = y + d$
- (7) $w = z * 2$

Leader Statements

Rules:

1. First statement is a leader.
2. Target of a jump is a leader.
3. Statement following a jump is a leader.

Leaders

Statement 1

Statement 4

Statement 6

Statement 7

Basic Blocks

B1

- (1) $x = a + b$
- (2) $y = x * c$
- (3) if $y > 50$ goto 6

B2

- (4) $z = y - d$
- (5) goto 7

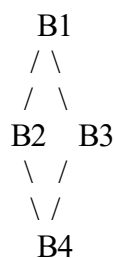
B3

- (6) $z = y + d$

B4

- (7) $w = z * 2$

Control Flow Graph



3. Construct the DAG for the following basic block:

$p = a + b$
 $q = a + b$
 $r = p * q$
 $s = a + b$
 $t = r + s$

Using the DAG, identify all **common sub-expressions**. Then simulate a simple code generator to produce optimized target code and explain the register allocation decisions at each step.

3A) DAG Construction

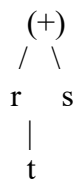
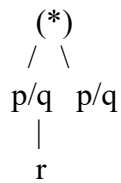
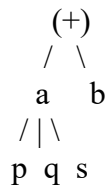
Basic Block

$p = a + b$
 $q = a + b$
 $r = p * q$

s = a + b

t = r + s

DAG



Common Sub-Expressions

a + b

appears in:

p = a + b

q = a + b

s = a + b

Computed only once.

Optimized Code

t1 = a + b

r = t1 * t1

t = r + t1

Target Code

MOV R0, a

ADD R0, b

MOV R1, R0

MUL R1, R0

ADD R1, R0

MOV t, R1

Register Allocation

R0 → (a+b)

R1 → r

R1 → t (final result)

4. For a target machine with two registers **R0** and **R1**, simulate the working of a **simple code generator** for the code:

t1 = a + b t2 =

c * d t3 = t1 -

t2 t4 = e + f t5

= t3 / t4

Show the **register descriptor** and **address descriptor** after each instruction is processed.

4A) Simple Code Generator

Three Address Code

t1 = a + b
t2 = c * d
t3 = t1 - t2
t4 = e + f
t5 = t3 / t4

Generated Code

LOAD R0,a
ADD R0,b

LOAD R1,c
MUL R1,d

SUB R0,R1

LOAD R1,e
ADD R1,f

DIV R0,R1
STORE t5,R0

Register Descriptor Table

Step	R	R
	0	1
t1=a+b	t1	-
t2=c*d	t1	t2
t3=t1-t2	t3	t2
t4=e+f	t3	t4
t5=t3/t4	t5	t4

Address Descriptor

Variable	Location
e	n
t1	R0
t2	R1
t3	R0
t4	R1
t5	R0

5. For a **RISC-style target machine** with instructions **LOAD, STORE, ADD, SUB, MUL, MOV**, simulate target code generation for the expression:

(a - b) * (c + d) - e

Show the complete sequence of target instructions generated. Also explain the **runtime storage management** and **stack frame layout** used during the computation.

Given the three-address code sequence:

```
t1 = a + b,  
t2 = t1 + 0,  
t3 = t2 * 1,  
if t3 == 0 goto L1,  
t4 = t3 + t3
```

**5A) Target Code
Generation for
RISC Machine
Expression**

$(a - b) * (c + d) - e$

Target

Instructions

LOAD R0, a

SUB R0, b

LOAD R1, c

ADD R1, d

MUL R0, R1

SUB R0, e

STORE RESULT,
R0

**Runtime Storage
Management**

- Variables stored in memory.
- Registers hold temporary results.
- Intermediate values remain in registers whenever possible.
- Final result stored back to memory.

**Stack Frame
Layout**

```
-----  
| Return Address |  
-----  
| Saved Registers|  
-----  
| Local Variables|  
-----  
| Temporaries   |  
-----  
| Parameters    |  
-----
```

Final Result

RESULT = $(a - b)$

$$* (c + d) - e$$

RUBRICS FOR CALCULATION

Simulation tools

Criteria	Weight age (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Conceptual Understanding	20%	Demonstrates strong understanding of underlying concepts in the simulation	Shows good understanding with minor gaps	Partial understanding; some misconceptions	Lacks understanding of key concepts
Model Design	25%	Develops accurate, well-structured simulation/model independently	Develops correct model with minor errors	Model is incomplete or requires guidance	Unable to develop a proper model
Tool Usage	20%	Uses simulation tools effectively with correct setup and execution	Uses tools with minor errors or guidance	Limited ability to use tools properly	Unable to use simulation tools
Analysis of Results	20%	Provides clear, logical analysis and meaningful interpretation of results	Analysis is reasonable with minor gaps	Superficial analysis; limited interpretation	Unable to analyze or interpret results
Documentation	15%	Presents results clearly with proper documentation and structured reporting	Documentation is adequate with minor issues	Poorly organized or incomplete documentation	No proper documentation or presentation